



THE PLAYBOOK · CONDENSED

Twenty-two rules for changing a CRM you can't afford to break.

The judgment calls behind a live HubSpot portal — from read-only diagnosis to clean decommission. Including the one rule I designed and never shipped.

Client names withheld by agreement. No outcome or ROI figures appear anywhere in this document, because none were ever measured.



Ian Sequeira

FRACTIONAL HUBSPOT & REVOPS

How to read this

These are the rules I actually apply, in the order I apply them: diagnose before touching anything, govern every write, use a model only where it earns its place, report the truth over the flattering number, ship new automation without letting it near your live records, and switch it all off cleanly when the work ends.

Most of these rules have shipped in client portals. A few have not, and those are labeled rather than quietly folded in with the rest — the label is the point. The full version of each rule, with the failure it prevents and an anonymized account of where it was applied, lives at the address in the footer.

SHIPPED IN PART	The rule holds, but not everything it describes is running.
DESIGNED, NEVER SHIPPED	Specified in a build plan and never deployed. The reasoning still holds.

DIAGNOSE

01 Diagnose before you build

The first deliverable is a fixed-scope, read-only audit of data hygiene, pipeline, coverage, and workflow cruft — and its findings, not assumptions or a preset roadmap, decide what gets built and in what order.

02 Surface the coverage gaps

A structural absence — a customer with no renewal deal, a record with no owner — is a finding to detect and hand back by name, not a defect the client should have to notice on their own.

03 Inventory workflows before building

Before you stack new automation on a portal, inventory every workflow it already runs and flag the archive candidates and the silently-inactive ones — you can't build cleanly on a layer nobody has read.

04 Baseline before you score

Before you build any scoring model, pull the current state of your existing deals and their associations read-only, so the model is designed against what the data actually shows — not against a guess.

GOVERN EVERY WRITE

05 Make every write opt-in

Every script that can change your CRM defaults to report-only — a write happens only when an operator deliberately opts that run in.

06 Fill blanks, never overwrite

An automated job writes to a CRM field only when it is empty, so a value a person entered by hand is never silently overwritten.

07 Stamp every write's source

Every automated write records which system produced it — a companion source field carrying a marker like model-and-date or the workflow that fired — so any value's origin is auditable and the rule about who may overwrite has something to read.

08 Let the first writer win

When a native workflow and an API job can both write the same field, each enrolls only while a companion source flag is still unset and stamps that flag on write — so the fast path and the slow path never overwrite each other.

09 Operationalize their own model

When a client already trusts a strategy model it authored itself — a territory map, a state-level scoring spreadsheet — operationalize that model into CRM properties and backfill it across the whole base in one governed, dry-run-first pass, rather than importing a scoring vendor's opinion the client never wrote.

SHIPPED IN PART

USE A MODEL SPARINGLY

10 Reserve the model for judgment

Let code do the reading, parsing, and writing; call the model only for the single judgment it alone can make — and run that call through a batch API at roughly half the cost of live requests.

11 Gate AI on confidence

When a model enriches your records, write only the labels it's confident about and route everything below the threshold to a human review queue — a blank field is recoverable, a wrong one silently corrupts every report built on it.

12 Mine rules from the model

Once a model has labeled your data, mine its output into deterministic rules — so cheap code handles the high-volume easy cases in real time and you pay the model only for the tail it alone can resolve.

13 Never re-pay for settled records

Read only the records that still need a decision and write each one once, so a re-run re-pays the model nothing for work already done.

SHIPPED IN PART

REPORT THE TRUTH

14 Gate stages on completed actions

A pipeline stage should be defined by something that verifiably happened — a meeting held, a proposal sent — not by rep intent like "proposal coming" or "verbal yes," because a stage that describes hope turns every forecast into a guess.

SHIPPED IN PART

15 Define the metric before building the report

Before building any report, agree the exact operational definition of each metric — what counts as held, as a demo, as a win, and which pipeline is excluded — because the intuitive definition usually undercounts real work.

16 Make notes countable

Real activity typed into free-text notes stays invisible to your dashboards — convert those notes into structured call and meeting engagements so the work is credited without changing how reps work.

17 Derive the blank field

A blank field is missing input, not missing information — when a value was never recorded, derive it from the record's own activity history rather than accepting the blank.

18 Snapshot what it won't keep

If the platform keeps no history of a score, snapshot it on a schedule yourself — trend and movement over time only exist if you record them.

SHIP WITHOUT BREAKING IT

19 Rehearse in production

Build new automation disabled, rehearse it end-to-end in your live portal behind an enrollment guard proven to keep every real record out, and leave it off until you sign off.

SHIPPED IN PART

20 Design for the trigger's blind spot

When a single event is an automation's only trigger, design a separate trip-wire report for the failure mode where that event never fires.

DESIGNED, NEVER SHIPPED

21 Monitor the jobs you ship

Run a daily health check on your own shipped automations, not just the client's data, so a silent failure surfaces in hours instead of when a report finally goes stale.

RETIRE CLEANLY

22 Leave nothing running

When an engagement ends, deliberately retire every automation you shipped and stop its triggers at the source — an unowned job left running is a liability, not a parting gift.

Start where every engagement starts.

If any of these named a problem you recognize, the first step is the one every engagement starts with — a free 30-minute read-only look at your actual portal, and a prioritized fix list that's yours to keep whether or not we work together.

isequeira251.github.io